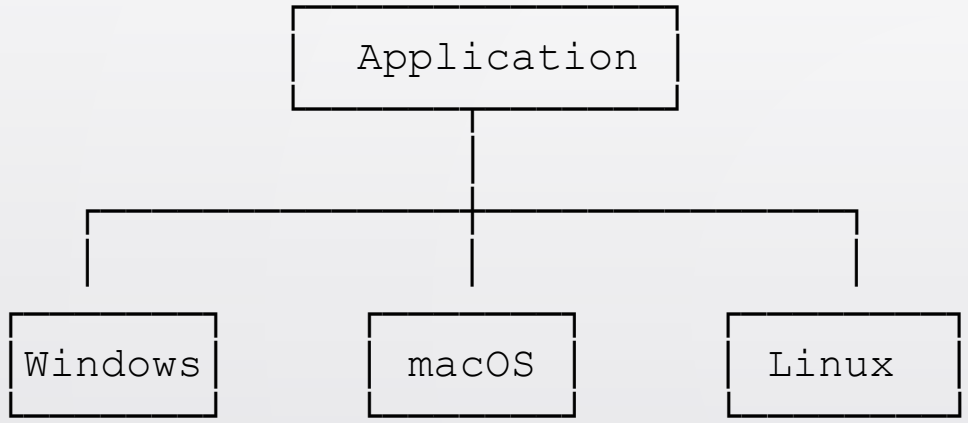
# Cross Platform Application Development

Introduction

# What is Cross-Platform Application Development?




**Concept:** Develop once → Run on multiple platforms

# What is Cross-Platform Application Development?

**Cross-platform development refers to:**

The ability to develop an application once and run it on multiple operating systems or devices with minimal modification.

**Examples of Platforms:** Windows, macOS, Linux, Android, iOS, Web browsers

**Examples of Devices:** PC, Laptop, Tablet, Mobile Phone

# Why Cross-Platform?

| Problem with Native | Benefits of Cross-Platform |
| --- | --- |
| • Separate codebase for each OS<br>• Higher development cost<br>• Longer maintenance cycle<br>• Technology dependency | • Code reusability<br>• Faster development<br>• Lower cost<br>• Easier maintenance<br>• Wider accessibility |

# Two Examples of Cross-Platform Approaches

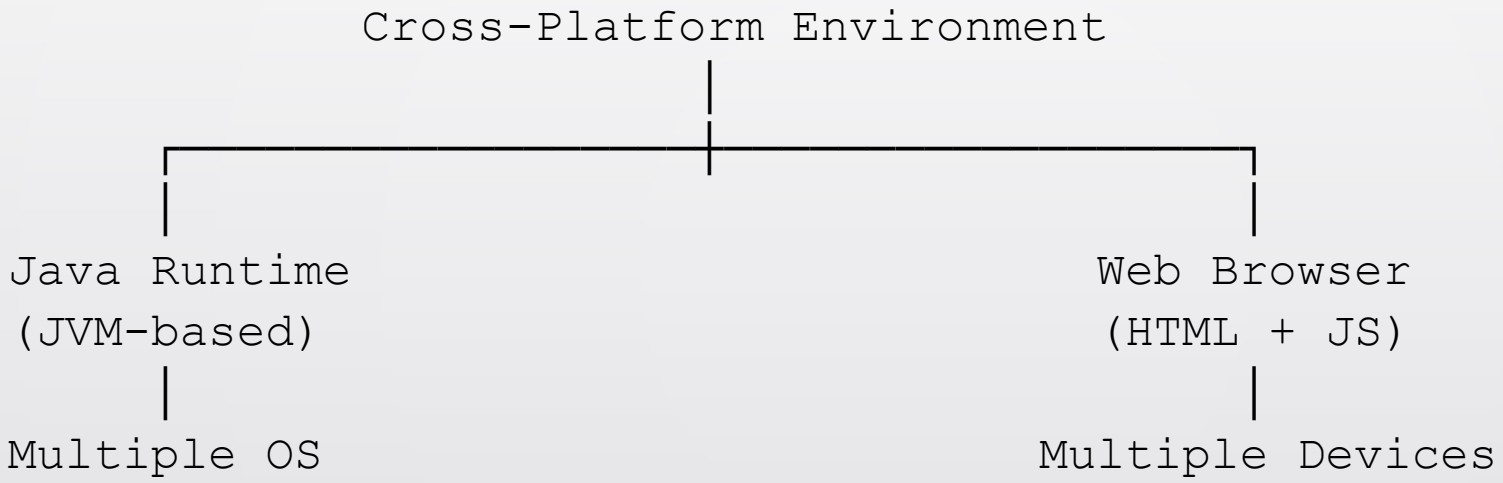
**Runtime-Based (e.g., Java)**

- Compile once
- Run on any OS with compatible runtime

**Browser-Based (Web Technology)**

- Runs inside web browser
- Platform-independent by design

# Two Examples of Cross-Platform Approaches



# Java as a Cross-Platform Technology

**Key Concept:** Write Once, Run Anywhere (WORA)

You have learnt this in Advanced Programming

**How It Works:**

- Java source code → Compiled into **Bytecode**
- Bytecode runs on **Java Virtual Machine (JVM)**
- JVM available on multiple operating system

**Architecture Flow:** Java Source → Java Compiler → Bytecode → JVM → OS

**Advantages:** Platform independence, Strong object-oriented support, Large ecosystem

# Web Technology as Cross-Platform Environment

**Core Technologies:**

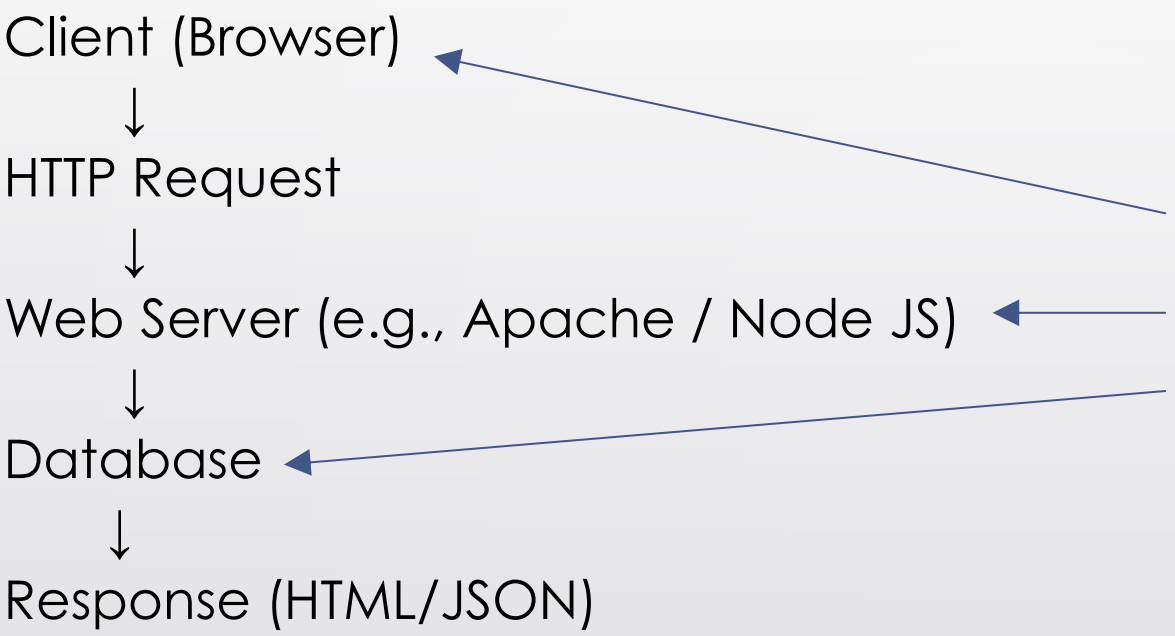
- HTML → Structure & Content
- CSS → Presentation
- JavaScript → Behaviour (Client side logic)
- Web Browser → Execution Environment
- Web Server → Backend Processing (Server side logic)




**Web is Naturally Cross-Platform:**

- Any device with a browser can run the application
- No OS-specific compilation required

# Web Application Architecture




**Key Components:**

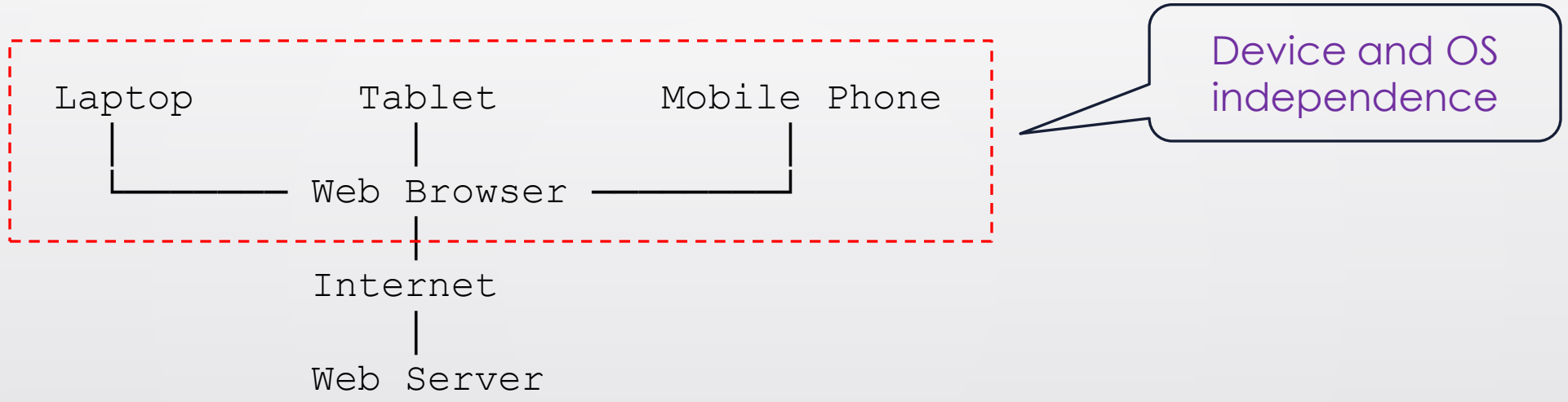
- Frontend (HTML + CSS + JavaScript)
- Backend (CGI, Java Servlet, PHP)
- Database (MySQL, PostgreSQL, Oracle, Firebase)

# Web Application Architecture

- 3-Tier Architecture View of Web Application



# How Browser Enables Cross-Platform




**Point:** As long as the device has a browser → Application runs.

# Evolution of Cross-Platform Environment

Command Line & Desktop Native Apps

↓

Java Cross-Platform Apps

↓

Web Applications (MPA – Multi Page Application)

↓

Single Page Applications (SPA)

↓

Cloud-Based Systems

Dynamic scripting languages (e.g., Perl, Python) are considered cross-platform because their interpreters run on multiple operating systems.

**Discussion:**
Why are modern systems moving toward browser-based architecture?

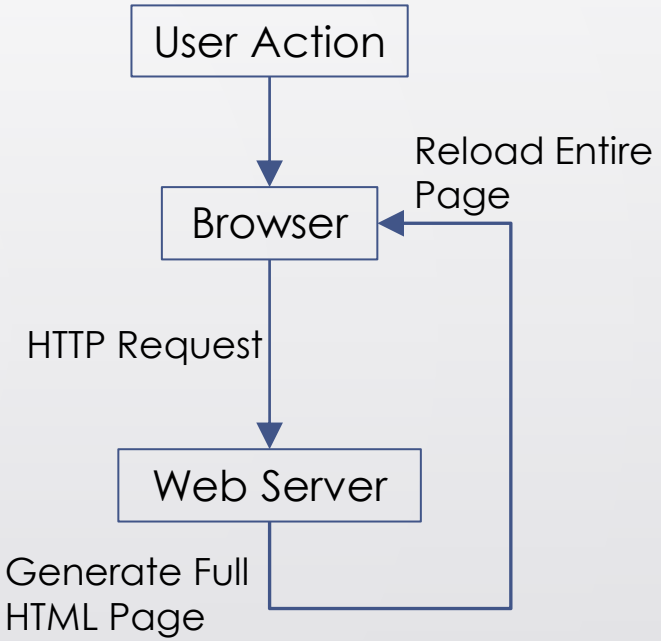
# MPA & SPA Web Application

| MPA | SPA |
| --- | --- |
| Multi-Page Application | Single-Page Application |
| Reloads entire page | Updates part of the page |
| Server-driven rendering | Client-driven rendering |

# MPA Web Application Architecture

**Key Characteristics**

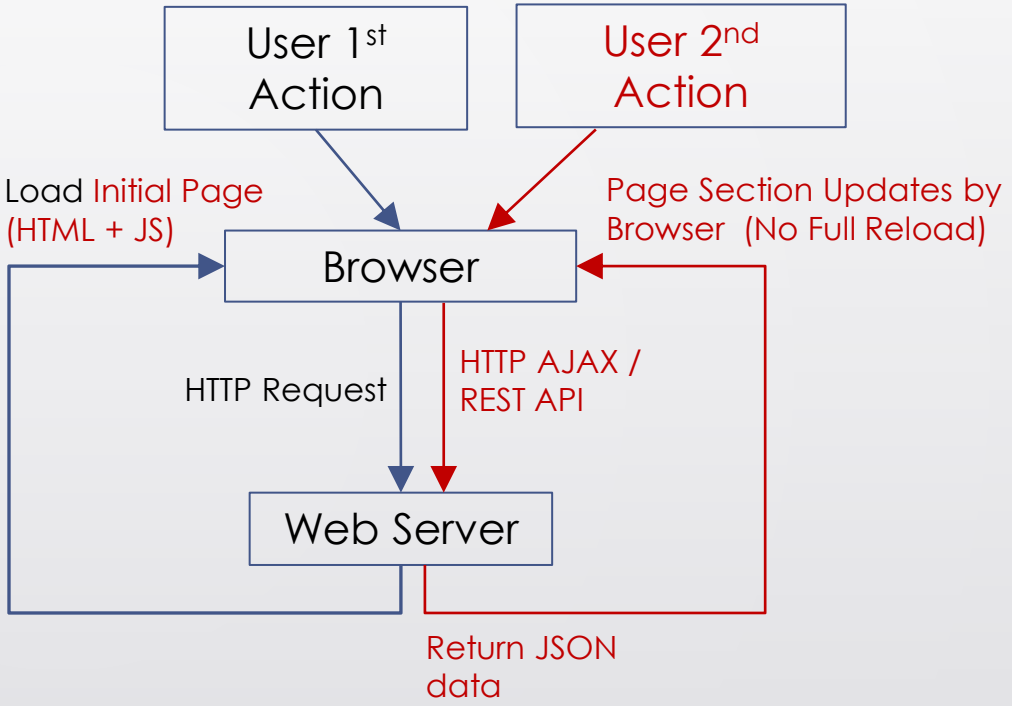
- Each action → new HTTP request
- Entire page reload
- Backend handles most rendering
- Traditional model (e.g., CGI, PHP, JSP)



# SPA Web Application Architecture

**Key Characteristics**

- One initial page load
- No full-page refresh
- Frontend handles UI rendering
- Backend provides data (API)



# MPA & SPA Technical Comparison

| Feature | MPA | SPA |
|---|---|---|
| **Page Reload** | Yes | No |
| **Rendering** | Server-side | Client-side |
| **User Experience** | Slower transitions | Smooth transitions |
| **Initial Load** | Faster | Slightly heavier |
| **Complexity** | Simpler | More complex |
| **Suitable For** | Content-based systems | Interactive systems |

## When to use MPA?

- Content-heavy website
- SEO-focused site
- Simple CRUD systems
- Lower frontend complexity

## When to use SPA?

- Highly interactive systems
- Dashboard / Admin panel
- Real-time updates Modern web applications
- API-driven architecture

# MPA & SPA Major Concepts

| MPA | SPA |
| --- | --- |
| Server-Centric - | Client-Centric |
| Server renders page | Browser renders page |
| Page-based navigation | Component-based dynamic rendering |
| | |
| | |

# Server-Centric Concept in MPA

**In a Multi-Page Application (MPA), the system is server-centric because:**

- The server handles most of the processing and page rendering.
- Each user request sends an HTTP request to the server.
- The server generates a completely new HTML page.
- The browser reloads the entire page.

# Client-Centric Concept in SPA

**In a Single-Page Application (SPA), the system is client-centric because:**

- The browser (client) handles UI rendering and navigation.
- The page loads once at the beginning.
- JavaScript dynamically updates parts of the page.
- The server mainly provides data (usually JSON via API).

# SPA and Mobile App Development

- It is possible to develop Hybrid Mobile App by embedding web technologies (HTML, CSS, JavaScript) inside a mobile app container
- SPA makes this possible because:
  - UI is already client-side rendered (Component-based UI)
  - Navigation is dynamic
  - Backend communication is API-based (REST API communication)

# Mobile App: Native vs Hybrid

**Native Mobile App**

Java/Kotlin (Android)/ Swift (iOS)

↓

Compiled separately

↓

Platform-specific apps

**Hybrid Mobile App**

HTML + CSS + JavaScript (SPA)

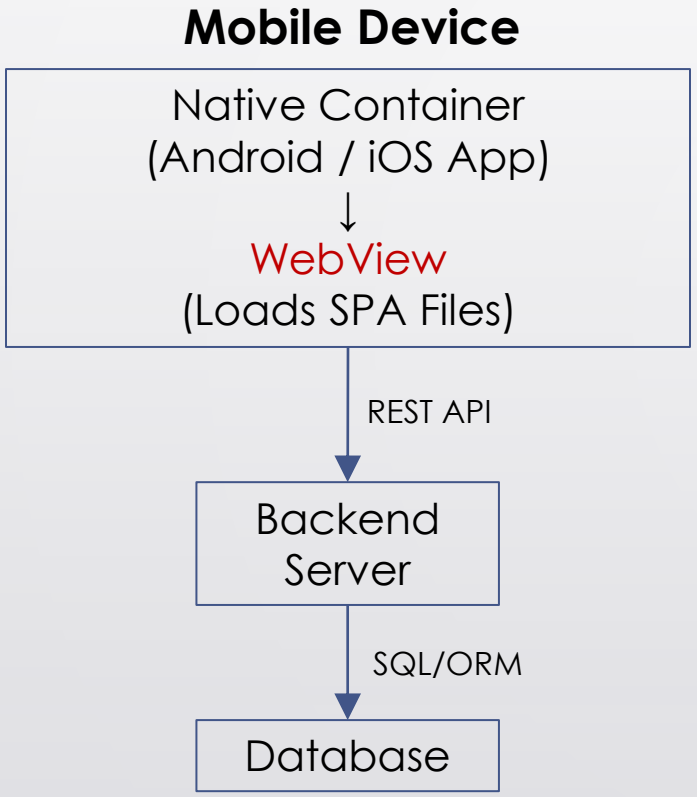
↓

WebView Container

↓

Android & iOS

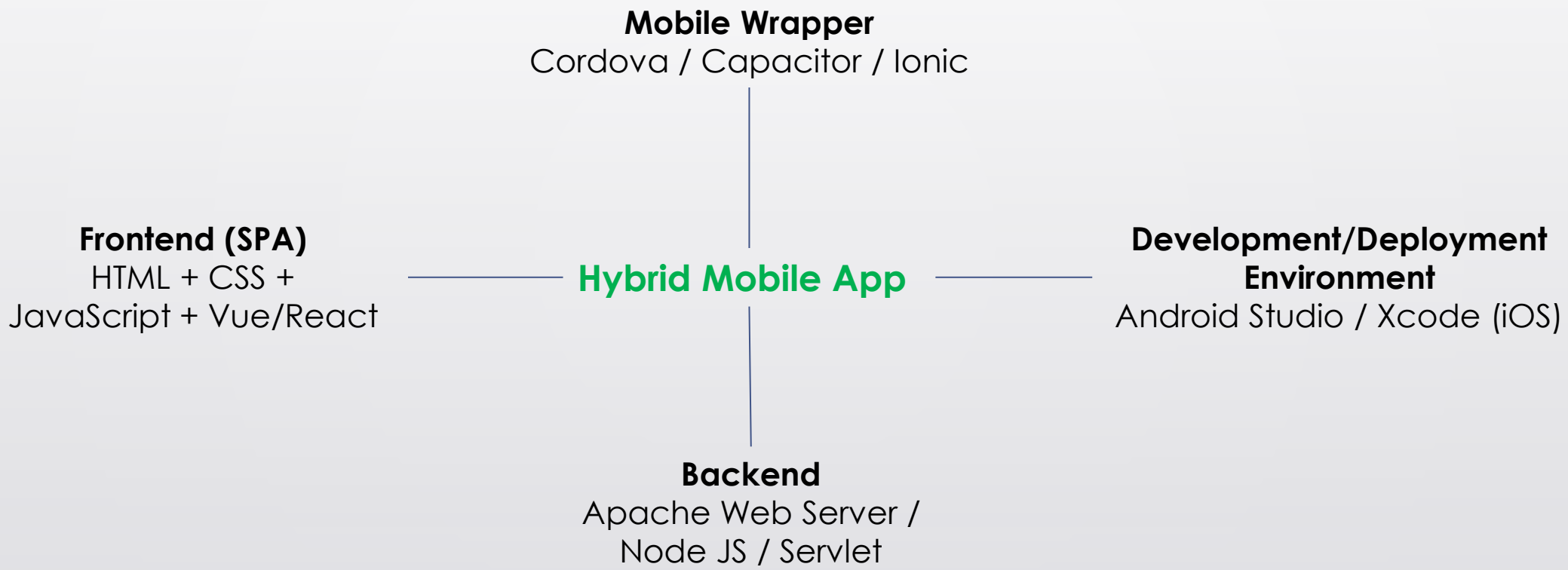
# Hybrid Mobile Architecture Using SPA




**Explanation:**

- The mobile app contains a WebView
- WebView loads the SPA (HTML + JS)
- SPA communicates with backend via API

# Hybrid Mobile Technology Stack



# Hybrid Mobile – Wrapper

**Main Roles of Wrappers (Cordova / Capacitor / Ionic):**

- Web-to-Native Bridge - connect JavaScript code to native device features, enable access to camera, GPS, file system, notification, etc.
- WebView Container - load and run the SPA inside a **native WebView** and allow HTML, CSS, and JavaScript to behave like a mobile app
- Plugin Management - provide plugin architecture and extend functionality using native APIs
- Cross-Platform Packaging Support - generate platform-specific projects for Android (via Android Studio) or iOS (via Xcode)

# Hybrid Mobile – Android Studio / Xcode

**Used as tools to support:**

- Project Container Management - generates Android/iOS project structure & wraps the SPA (HTML, CSS, JavaScript) inside a **WebView**
- Build & Compilation - compiles the hybrid app into an **APK / AAB** (Android) or **IPA** (iOS), prepares the app for installation on Android/iOS devices
- Device Testing & Emulator - provides Android/iOS Emulator & allows debugging on real devices
- Native Plugin Integration - connects web code to Android/iOS native features (Camera, GPS, Storage, Notification)

# Hybrid Mobile Technology Architecture

